SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: ITA 0603 COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO5: *Introduction – NumPy Arrays – NumPy Basics- Math – Random – Indexing – Filtering – Statistics – Aggregation – saving data – Data analysis with Pandas – DataFrame – Combining – Indexing – File I/O – Grouping – Filtering – Sorting – Plotting (BL 3)*

Assessment Tool Weightage: 40%

QUESTIONS: 15 **Total Marks:25**

Annexure A

Constraint Based Problem Solving

Code of Conduct:

I X. Joshua (192312188) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
X. Joshua

Annexure A

Short Answer Test

1. You are given a large dataset of patient health records stored in a CSV file, containing missing values, inconsistent formats, and outliers. Using NumPy and Pandas, design a constraint-based data cleaning pipeline that ensures: (i) no missing values remain, (ii) all numerical values fall within medically valid ranges, and (iii) categorical fields are standardized. Explain how you will use indexing, filtering, and aggregation functions to enforce these constraints while preserving maximum data integrity.

Answer:

A constraint-based data cleaning pipeline can be designed using **Pandas and NumPy**. First, the CSV file is loaded into a DataFrame and its columns and data types are inspected.

```
import pandas as pd
import numpy as np

df = pd.read_csv("patients.csv")

df.columns = df.columns.str.strip().str.lower()

df["age"] = pd.to_numeric(df["age"], errors="coerce")
df["blood_pressure"] = pd.to_numeric(
    df["blood_pressure"], errors="coerce"
)
```

Invalid numerical values can be identified using Boolean filtering. For example, age can be restricted to 0–120 and blood pressure can be restricted to a medically reasonable range.

```
df.loc[(df["age"] < 0) | (df["age"] > 120), "age"] = np.nan

df.loc[
    (df["blood_pressure"] < 50) |
    (df["blood_pressure"] > 250),
    "blood_pressure"
] = np.nan
```

Missing numerical values can be replaced using aggregation functions such as median().

```
df["age"] = df["age"].fillna(df["age"].median())
df["blood_pressure"] = df["blood_pressure"].fillna(
    df["blood_pressure"].median()
)
```

Categorical values can be standardized using string functions and replacement.

```
df["gender"] = df["gender"].str.strip().str.lower()

df["gender"] = df["gender"].replace({
    "m": "male",
    "f": "female"
})

df["gender"] = df["gender"].fillna("unknown")
```

Finally, constraints are verified using isnull(), unique(), value_counts(), and filtering. This preserves data integrity because valid records are retained while only missing, inconsistent, or invalid values are corrected.

2. A system with limited memory must process a massive numerical dataset using NumPy arrays. You are required to perform mathematical operations, random sampling, and statistical analysis without exceeding memory limits. Propose a solution that uses efficient array creation, slicing, and in-place operations. Discuss how indexing and aggregation can be optimized to minimize memory usage, and explain debugging strategies if memory overflow or performance bottlenecks occur.

Answer:

For a massive numerical dataset, NumPy arrays should be created with an appropriate data type such as float32 instead of float64 when high precision is not required.

```
import numpy as np

data = np.random.random(10_000_000).astype(np.float32)
```

Slicing can be used to access portions of an array without creating unnecessary copies.

```
sample = data[::10]
```

In-place operations such as *=, +=, and /= reduce additional memory allocation.

```
data *= 2
```

Statistical aggregation can be directly performed on the array.

```
mean = data.mean()
maximum = data.max()
minimum = data.min()
```

Indexing should select only the required elements. Random sampling can also be performed using a limited number of indices instead of copying the complete dataset.

Memory usage can be monitored using:

```
print(data.nbytes)
```

If memory overflow occurs, the data type can be reduced, the dataset can be processed in chunks, and unnecessary copies should be avoided. Performance bottlenecks can be debugged by measuring execution time and identifying repeated calculations or inefficient loops.

3. Two large Pandas DataFrames containing customer transaction and demographic data must be combined. However, constraints include: (i) no duplication of customer IDs, (ii) consistent indexing after merging, and (iii) preservation of all valid records. Describe how you will perform the combining operation, validate constraints using filtering and grouping, and ensure correctness through sorting and indexing techniques. Include steps to debug mismatched or missing entries.

Answer:

The two DataFrames should be combined using the common customer_id column. Before merging, duplicate customer IDs should be checked.

```
print(demographics["customer_id"].duplicated().sum())
```

The DataFrames can then be merged using an outer join to preserve all valid records.

```
merged = pd.merge(
    transactions,
    demographics,
    on="customer_id",
    how="outer",
    validate="many_to_one"
)
```

Grouping can be used to identify unexpected duplicate customer IDs.

```
print(merged.groupby("customer_id").size())
```

Missing entries can be identified using filtering.

print(merged[merged["customer_name"].isna()])
The final DataFrame should be sorted and its index reset.
merged = merged.sort_values("customer_id")
merged = merged.reset_index(drop=True)
The original and merged customer IDs should be compared using isin() to identify unmatched records. This approach ensures that duplicate IDs are detected, all valid records are preserved, and the final DataFrame has a consistent index.

---

4. You are tasked with computing statistical measures (mean, median, variance) on a dataset using NumPy and Pandas, but with constraints on both computational efficiency and numerical accuracy. Explain how you will implement aggregation and statistical functions while avoiding precision loss and redundant computations. Discuss how filtering and indexing can be used to exclude irrelevant data points, and how you would debug incorrect statistical outputs.
Answer:
NumPy and Pandas provide efficient aggregation functions for calculating statistical measures.
```
import pandas as pd

data = pd.Series([10, 20, 30, 40, 50])

mean = data.mean()
median = data.median()
variance = data.var()
```
Irrelevant or invalid values can first be removed using filtering.
```
valid = data[(data >= 0) & (data <= 100)]
```
The required statistics can then be calculated on the filtered data. float64 can be used when higher numerical precision is required.
Missing values should be handled appropriately using Pandas functions such as mean() and median().
Redundant computations should be avoided by calculating each statistic only when required and storing the result.
If the output is incorrect, the dataset should be checked for missing values, incorrect data types, outliers, and unexpected ranges. The number of valid observations should also be verified. A small sample can be calculated manually to confirm the result.

---

5. Design a complete data analysis pipeline that reads multiple data files, processes them using Pandas DataFrames, and generates visualizations. Constraints include: (i) efficient file I/O operations, (ii) correct grouping and sorting of data, (iii) filtering based on user-defined conditions, and (iv) meaningful plotting outputs. Explain how each stage (loading, processing, aggregation, and plotting) will be implemented and optimized, along with debugging steps for handling corrupted files or inconsistent data formats.
Answer:
A complete data analysis pipeline consists of **data loading, cleaning, filtering, aggregation, sorting, and visualization**.
```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("sales.csv")

df.columns = df.columns.str.strip().str.lower()
```

```
df["sales"] = pd.to_numeric(
    df["sales"], errors="coerce"
)

df = df.dropna(subset=["sales"])
```
User-defined conditions can be applied using filtering.
```
filtered = df[df["sales"] > 1000]
```
Data can be grouped and aggregated using groupby().
```
summary = (
    filtered.groupby("category")["sales"]
    .sum()
    .sort_values(ascending=False)
)
```
The results can be visualized using Matplotlib.
```
summary.plot(kind="bar")

plt.xlabel("Category")
plt.ylabel("Total Sales")
plt.title("Sales by Category")
plt.show()
```
For multiple files, pd.concat() can combine compatible DataFrames. Only required columns should be loaded when possible to improve memory efficiency. Corrupted files can be detected using exception handling, while info(), dtypes(), isna(), and sample records can be used to debug inconsistent data.

---

6. You are given a large dataset of financial transactions containing duplicate records, inconsistent timestamps, and invalid entries. Design a data cleaning pipeline using NumPy and Pandas with constraints: (i) remove duplicates without losing valid data, (ii) standardize time formats, and (iii) filter out invalid transactions. Explain how indexing, grouping, and sorting ensure data integrity.

Answer:

The dataset should first be loaded and duplicate records should be removed.
```
df = df.drop_duplicates()
```
Timestamps can be standardized using pd.to_datetime().
```
df["timestamp"] = pd.to_datetime(
    df["timestamp"],
    errors="coerce"
)
```
Invalid timestamps can be removed.
```
df = df.dropna(subset=["timestamp"])
```
Invalid financial transactions can be filtered.
```
df = df[
    (df["amount"] > 0) &
    (df["account_id"].notna())
]
```
Duplicate transaction IDs can be identified using grouping.
```
df.groupby("transaction_id").size()
```
Finally, records can be sorted by timestamp and the index can be reset.
```
df = df.sort_values("timestamp")
```

df = df.reset_index(drop=True)
These operations ensure that duplicate records are removed, timestamps are standardized, invalid transactions are excluded, and the final dataset maintains proper indexing and chronological order.

---

7. A real-time sensor system generates streaming data that must be processed under strict latency constraints. Using NumPy arrays, design a solution that performs filtering, normalization, and anomaly detection efficiently. Ensure minimal computation delay and memory usage. Discuss how slicing, vectorization, and aggregation functions can be optimized, and explain debugging techniques for delayed processing or incorrect outputs.
Answer:
NumPy vectorization can be used to process sensor data efficiently without Python loops.

```
import numpy as np

data = np.array(
   [20, 21, 22, 100, 23],
   dtype=np.float32
)

valid = data[(data >= 0) & (data <= 50)]

normalized = (
   (valid - valid.mean()) /
   valid.std()
)

anomalies = valid[
   np.abs(normalized) > 2
]
```

Slicing can process only the latest sensor readings instead of the complete history. Vectorized operations reduce processing time, while float32 reduces memory consumption.
Aggregation functions such as mean(), std(), min(), and max() can be used for real-time statistics.
If processing is delayed, execution time should be measured and unnecessary copies or loops should be removed. Incorrect results can be debugged by checking array shape, data type, sensor ranges, missing values, and a small set of manually verified readings.

---

8. You are required to merge multiple large datasets containing product information from different sources. Constraints include: (i) resolving conflicting attribute values, (ii) maintaining consistent schema, and (iii) avoiding data loss. Describe how you will use Pandas merging, grouping, and filtering techniques to enforce constraints and validate the final dataset.
Answer:
Before merging, column names and data types should be standardized.

```
df1.columns = df1.columns.str.strip().str.lower()
df2.columns = df2.columns.str.strip().str.lower()
```

The datasets can then be merged using product_id.

```
merged = pd.merge(
   df1,
   df2,
```

```
    on="product_id",
    how="outer",
    suffixes=("_source1", "_source2")
)
```
An outer merge ensures that records from both sources are preserved.
Conflicting values can be identified using filtering.
```
conflicts = merged[
    merged["price_source1"].notna() &
    merged["price_source2"].notna() &
    (merged["price_source1"] !=
     merged["price_source2"])
]
```
A priority rule can be used to resolve conflicts.
```
merged["price"] = (
    merged["price_source1"]
    .fillna(merged["price_source2"])
)
```
Grouping by product_id helps detect duplicates. Finally, missing values, schema consistency, duplicate IDs, and record counts should be validated.

---

9. A dataset contains millions of records with mixed data types and must be transformed into a structured format suitable for machine learning. Constraints include: (i) efficient type conversion, (ii) handling missing and categorical data, and (iii) ensuring consistent indexing. Explain how you will use Pandas operations and NumPy functions to preprocess data while maintaining computational efficiency.

Answer:

The dataset should first be inspected using info() and isnull().
```
df.info()
print(df.isnull().sum())
```
Numerical columns should be converted using pd.to_numeric().
```
df["age"] = pd.to_numeric(
    df["age"],
    errors="coerce"
)
```
Missing numerical values can be replaced using the median.
```
df["age"] = df["age"].fillna(
    df["age"].median()
)
```
Categorical values can be encoded using get_dummies().
```
df = pd.get_dummies(
    df,
    columns=["gender", "city"]
)
```
NumPy can efficiently convert numerical data.
```
X = df["age"].to_numpy(dtype=np.float32)
```
Finally, indexing can be standardized.
```
df = df.reset_index(drop=True)
```
Vectorized Pandas and NumPy operations should be preferred over loops because they are more efficient for millions of records.

---

10. You need to compute rolling statistics (moving average, rolling variance) on a time-series dataset under constraints of high performance and accuracy. Describe how to implement rolling window operations using Pandas and NumPy, ensuring efficient indexing and minimal redundancy. Discuss debugging strategies for incorrect window alignment and performance bottlenecks.

Answer:

The time-series data should first be converted to a proper datetime format and sorted.

```
df["timestamp"] = pd.to_datetime(
    df["timestamp"]
)

df = df.sort_values("timestamp")
```

Pandas rolling() can calculate moving statistics.

```
df["moving_average"] = (
    df["value"].rolling(window=7).mean()
)

df["rolling_variance"] = (
    df["value"].rolling(window=7).var()
)
```

The rolling window contains the specified number of observations. Correct indexing and chronological sorting are important for accurate results.

Debugging should include checking the first few rows, verifying timestamp order, checking window size, and manually calculating a small sample.

Performance can be improved by selecting only required columns and avoiding repeated rolling operations. NumPy can also be used when customized high-performance numerical calculations are required.

---

11. You are given a large healthcare dataset with mixed structured and semi-structured data (CSV and JSON formats). Design a preprocessing pipeline using NumPy and Pandas with constraints: (i) unify different data formats into a single schema, (ii) handle nested JSON fields efficiently, and (iii) ensure no data inconsistency after transformation. Explain how indexing, normalization, and aggregation techniques are used, and describe debugging strategies for schema mismatches.

Answer:

CSV and JSON files can be loaded into Pandas DataFrames.

```
import pandas as pd

csv_df = pd.read_csv("patients.csv")
json_df = pd.read_json("patients.json")
```

Nested JSON data can be converted into tabular form using json_normalize().

```
json_df = pd.json_normalize(
    json_df.to_dict("records")
)
```

Column names should be standardized.

```
csv_df.columns = (
    csv_df.columns.str.strip().str.lower()
)

json_df.columns = (
```

```
    json_df.columns.str.strip().str.lower()
)
```
The DataFrames can then be merged or concatenated depending on their structure. Indexing can be standardized using:
```
df = df.reset_index(drop=True)
```
Grouping can detect duplicate patient records.
```
df.groupby("patient_id").size()
```
Schema mismatches can be debugged using columns, dtypes, info(), and comparisons of column names and data types. Missing values and duplicate records should also be checked after transformation.

---

12. A financial analytics system must compute real-time risk metrics from streaming stock market data under strict latency constraints. Using NumPy, design a solution that performs vectorized computations for returns, volatility, and correlations. Ensure minimal memory usage and fast execution. Explain how slicing, broadcasting, and aggregation functions are optimized, and discuss debugging techniques for incorrect calculations or delayed outputs.

Answer:

NumPy can calculate financial risk metrics using vectorized operations.
```
import numpy as np

prices = np.array(
    [100, 102, 101, 105, 107],
    dtype=np.float64
)

returns = prices[1:] / prices[:-1] - 1

volatility = np.std(returns)
```
Slicing is used to compare consecutive prices without explicit loops. Broadcasting can perform calculations across multiple assets efficiently.
Correlation can be calculated using:
```
correlation = np.corrcoef(returns)
```
Aggregation functions such as mean() and std() calculate important risk statistics quickly.
Memory usage can be reduced by processing only the required recent data window and avoiding unnecessary copies.
Debugging should include checking array shapes, price ordering, missing values, and manually calculating a few returns. Latency can be improved by eliminating loops and redundant computations.

---

13. You are tasked with cleaning a large dataset of social media posts containing text, timestamps, and user metadata. Constraints include: (i) removal of duplicate and spam entries, (ii) normalization of text data, and (iii) consistent timestamp formatting. Describe how you will use Pandas operations such as filtering, grouping, and string processing to enforce these constraints, and explain debugging strategies for handling noisy and inconsistent data.

Answer:

Duplicate records can first be removed using drop_duplicates().
```
df = df.drop_duplicates()
```
Text can be normalized using Pandas string functions.

```
df["text"] = (
    df["text"]
    .str.lower()
    .str.strip()
)
```
Timestamps can be standardized.
```
df["timestamp"] = pd.to_datetime(
    df["timestamp"],
    errors="coerce"
)
```
Posts containing spam keywords or excessive links can be filtered.
```
df = df[df["text"].notna()]
```
Duplicate posts can be identified using multiple columns.
```
df = df.drop_duplicates(
    subset=["user_id", "text", "timestamp"]
)
```
Grouping can identify users producing unusually large numbers of posts.
```
df.groupby("user_id").size()
```
Debugging involves checking null values, unique text formats, timestamp formats, duplicate counts, and suspicious user activity.

---

14. A machine learning pipeline requires feature extraction from a high-dimensional dataset under constraints of computational efficiency and scalability. Using NumPy and Pandas, design a solution that performs feature selection, normalization, and dimensionality reduction. Explain how indexing, vectorization, and aggregation are used to optimize performance, and describe debugging steps for detecting redundant or irrelevant features.

Answer:

First, numerical features can be selected from the DataFrame.
```
numeric = df.select_dtypes(
    include=np.number
)
```
Correlation can be used to detect redundant features.
```
correlation = numeric.corr()
```
Required features can then be selected using indexing.
```
X = df[
    ["age", "income", "score"]
]
```
NumPy can perform efficient normalization.
```
X = X.to_numpy(
    dtype=np.float32
)

X_normalized = (
    X - X.mean(axis=0)
) / X.std(axis=0)
```
Vectorization avoids slow Python loops, while aggregation functions calculate statistics such as mean, variance, and correlation.

Dimensionality reduction techniques such as PCA can be applied when the number of features is very large.

Debugging should check feature shape, missing values, zero-variance features, highly correlated features, and whether important information has been removed.

---

15. You are required to integrate and analyze data from multiple IoT devices with varying sampling rates and missing timestamps. Constraints include: (i) aligning time-series data correctly, (ii) handling missing intervals without distortion, and (iii) maintaining computational efficiency. Explain how you will use Pandas time-series functions, reindexing, interpolation, and aggregation to enforce these constraints, and discuss debugging techniques for misaligned or inconsistent time-series data.

Answer:

IoT timestamps should first be converted to Pandas datetime objects.

```
df["timestamp"] = pd.to_datetime(
    df["timestamp"]
)

df = df.set_index("timestamp")
df = df.sort_index()
```

Different sampling rates can be aligned using resampling.

```
aligned = df.resample("1min").mean()
```

Missing values can be handled using interpolation when the missing interval is short.

```
aligned = aligned.interpolate()
```

Reindexing can ensure that all devices use a common time index. Aggregation functions such as mean(), min(), and max() can summarize readings within each time interval.

For long missing intervals, interpolation should be avoided because it may create misleading values. Such values should remain missing or be handled according to the application requirements.

Debugging should include checking duplicate timestamps, missing timestamps, sampling frequencies, timezone differences, and the final aligned index. Sorting and inspecting a small time range can help identify misalignment problems.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**15**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**15**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand